

JAVA 程序设计



授课人：何毅
办公室：1909



第13讲 内容向导

13.1 异常处理概述

13.2 异常处理



13.1 异常处理概述

- 程序运行过程中，如果出现一个不可能执行的操作，就会出现运行时错误。运行时错误会引起异常。
- 异常：程序运行过程中出现的事件，它中断正常的程序控制流。没有异常处理代码的程序可能非正常结束，引起严重问题。
- Java给程序员提供了处理运行时错误的功能，这称为异常处理。



➤ 语法错

如果程序编译不通过，说明程序中存在语法错误。

语法错误是由于违反语法规则产生的。例如：标识符未声明、括号不匹配、语句末尾缺少分号等。

处理方法：Java编译器能发现语法错误，并给出错误的位置和性质。程序员必须改正这些语法错误，才能重新编译程序。

例： `string string="Tom";`



➤ 语义错

- 如果编译通过，但程序在运行时因为出现错误而被迫终止运行，则称这种错误为异常。例如：除数为0、访问数组下标越界等
- Java解释器在运行时能发现异常，一旦发现异常，**Java**将停止程序运行，并给出异常的位置和性质。
- 处理方法：若编程时考虑周全，有些异常是可以避免的。还有一些异常是不可预知的，不能在程序中事先处理，如打开文件不存在等，这些错误的发生不能由程序本身所控制，因此必须进行异常处理。



➤ 逻辑错

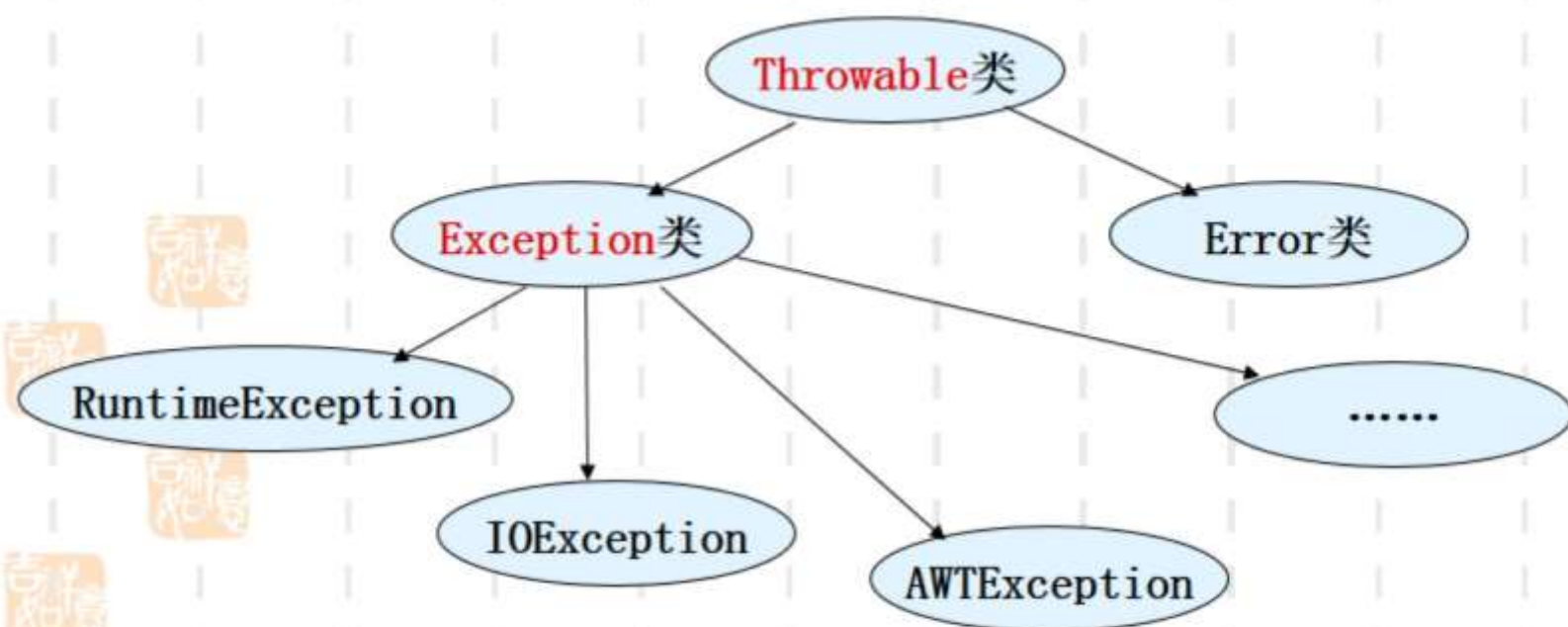
如果程序编译通过，也能正常运行，但运行结果与期望不相符，这说明程序中存在逻辑错误。例如：由于分支语句中的判断条件设置错误，导致某个数据参与到不正确的分支中进行计算。逻辑错往往是由于程序的设计思路不正确而造成的。

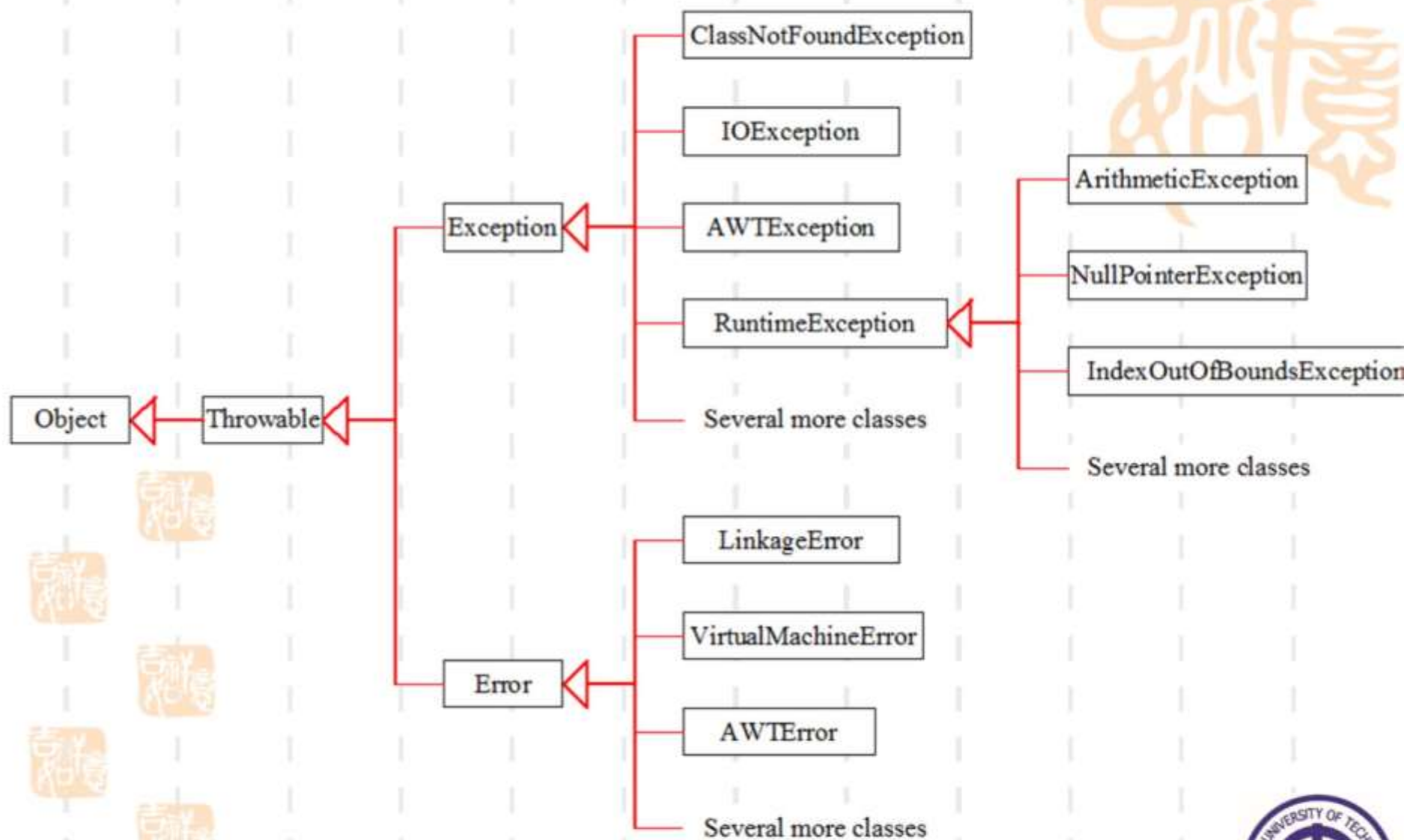
处理方法：系统是不能自动检查出逻辑错误的，只能靠编程人员凭经验分析并改错。



➤异常 (Exception)

程序执行过程中发生的中断了正常指令流的事件





- 例如，下面的程序会非正常终止，因为除数为0，引起数值计算错误。

```
public class test
{
    public static void main(String[] args)
    {
        System.out.println(3/0);
    }
}
```



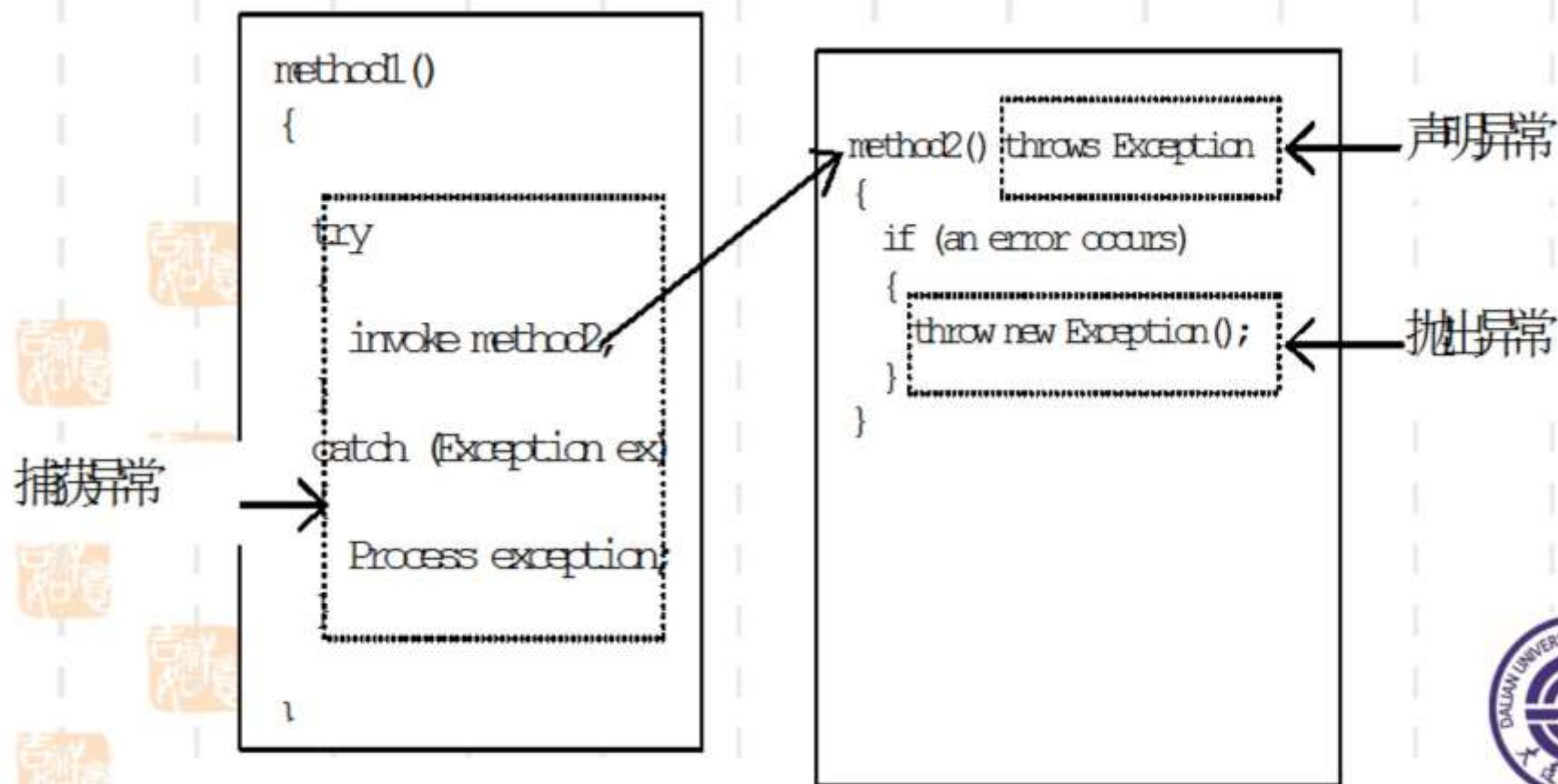
- 造成运行时错误的原因很多。比如，用户可能键入一个无效的输入值，程序可能试图打开一个不存在的文件或试图访问一个越界的数组元素等。当出现一个运行时错误时，Java就会产生一个异常。
- Java异常是Throwable子类的一个实例。
- Throwable包含在java.lang中，它的子类在不同的包中。与图形相关的错误包含在java.awt中，数值异常包含在java.lang中。扩展Throwable或它的子类，就能创建自己的异常类。



13.2 异常处理

Java的异常处理基于三种操作：

声明异常、抛出异常和捕获异常。



- 一般来说，每一个方法都必须在方法头说明它可能遇到的异常类型，这一过程称为声明异常。
- 在Java中，当前执行的语句必然属于某个方法。当一个语句引起错误时，含有这个语句的方法就会创建一个异常对象并将它传递给系统。异常对象包含有关异常的信息，包括它的类型和出错时程序的状态。这一过程称为抛出异常。
- 一个方法抛出异常后，Java运行系统开始寻找用来处理错误的代码。处理错误的代码称为异常处理器，从当前的方法开始，通过方法调用链向后搜索，查找这一代码。处理器必须与抛出的异常类型相匹配，如果未发现异常处理器，程序就会终止。这个寻找异常处理器的过程称为捕获异常。



➤ 声明异常

声明异常就是告诉编译器在执行方法的过程中可能出现的错误

➤ 在方法声明中使用关键字**throws**可以声明异常，如：

```
public void myMethod() throws IOException
```

```
public void myMethod()  
    throws IOException, OtherException
```

➤ 在方法头中用关键字**throws**进行声明！

➤ 注意：声明了抛出异常的方法未必会真的抛出异常！



➤ 抛出异常

在声明了异常的方法中，如果出现异常，就可以抛出一个异常对象。

例如：

```
throw new TheException();
```

或

```
TheException e = new TheException();  
throw e;
```

➤ 注意：不要和throws弄混！

➤ 注意：一个方法只能抛出在方法声明中声明的异常以及异常Error、RuntimeException和以上这些异常的子类。




```
public double divide(double r1, double r2)
    throws Exception
{
    if (r2 == 0)
    {
        throw new Exception("denominator
            cannot be zero");
    }
    return r1/r2;
}
```



- 当调用一个显式声明异常的方法时，必须使用try-catch块括住抛出异常的语句。

```
try
{
    statements;
}
catch (Exception1 e)
{ handler for exception1 }
catch (Exception2 e)
{ handler for exception2 }

...

catch (ExceptionN e)
{ handler for exceptionN }
```



```
class compute{
public double divide(double r1,double r2) throws Exception
{
    if (r2 == 0){
        throw new Exception("denominator cannot be zero");
    }
    return r1/r2;
}
}

public class qwer{
public static void main(String[] args){
    compute c=new compute();
    double result=0.0;
    try{
        result=c.divide(10, 0);
        System.out.println(result);
    }catch(Exception e){
        System.out.println(e.getMessage());
    }
}}
```





- `public String getMessage()`
返回Throwable对象的详细信息
- `public String toString()`
返回Throwable对象的简要描述。
- `public void printStackTrace()`
在控制台上打印Throwable对象和它的追踪信息。



➤ 对一段可能抛出大量不同类型的异常的程序来说，逐个捕获处理异常可能会较繁琐，而仅仅捕获它们父类的异常则会较方便。实际使用中，需要避免捕获最一般的异常类型（即类Exception），因为捕获最一般的异常使得用户往往不能确切地判断异常的具体类型并作出相应的处理。

➤ 若程序中有多个catch语句，要注意它们的顺序，正确的做法是先捕获最特殊的异常（子类异常），之后逐渐一般化（父类异常）。



➤ 例如:

```
try { ... }  
catch (Exception e) {  
    System.out.println("The error is: "+e.getMessage());  
}  
catch (IOException e) {  
    System.out.println("The error is: "+e);  
}  
catch (FileNotFoundException e) {  
    System.out.println("The error is: "+e.getMessage());  
}
```

➤ 后两个catch捕获语句会始终匹配不到，编译时会提示出错。所以说，捕获异常的顺序是和catch语句顺序相关的，要注意它的用法。





```
public class less {  
    public static void main(String args[])  
    {  
        int a,b,c;a=67; b=0;  
        try{  
            c=a/b;  
            System.out.println(a+"/"+b+"="+c);  
            int x[]=new int[-5];  
        }  
        catch(NegativeArraySizeException e)  
        {System.out.println("exception:"+e.getMessage());  
        e.printStackTrace();  
        }  
        catch(ArithmeticException e)  
        { System.out.println("b=0:"+e.getMessage());}  
        finally  
        { System.out.println("end");}}}
```



- 有时不论异常是否出现或者是否被捕获，都希望执行某些代码。Java有一个finally子句可以用来达到这一目的。
- finally子句的语法如下所示：

```
try {  
    statement1; //throw a exception  
    statement2;  
    .....  
}  
catch (TheException e) {  
    Process exception;  
}  
finally {  
    finalStatements;  
}  
otherStatements;
```





```
try {
```

有异常抛出的函数调用;

```
}
```

```
catch (捕获的异常类 变量) {
```

异常处理

```
}
```

```
catch (捕获的异常类 变量) {
```

异常处理

```
}
```

```
finally {
```

无论有无异常抛出, 都执行这里的语句;
主要用于资源释放;

```
}
```



课堂测试1



- 编写程序声明一个异常处理，用于判断数组元素是否为负数；
- 编写步骤：
 - 1.声明**D**类，包含一数组（**10**个元素），构造函数使得数组元素为随机数；
 - 2.编写一个方法**output**负责输出元素，当元素为负数时输出异常；
 - 3.编写执行主类测试。





```
import java.util.Random;
class D{
int d[]=new int[10];
Random rand=new Random();
D(){
for(int i=0;i<d.length;i++)
d[i]=(int)rand.nextInt(100)-50;
}
void output()throws Exception{
for(int i=0;i<d.length;i++)
if(d[i]>=0)
System.out.println(d[i]+" ");
else
throw new Exception(d[i]+"是负数，抛出异常");
}}
public class CollectionDemo{
public static void main(String args[]){
D d1=new D();
try{
d1.output();
}catch(Exception e){
System.out.println(e.getMessage());
}}}
```



复习与预习

复习

1.异常处理概述

2.异常处理

预习

多线程

